AWS Lambda

AWS Lambda

> **Lambda is AWS's "serverless function" service.**
>
> It lets you **run your code (Python, etc.) without managing any servers**.

🧠 "Just give AWS a small function. They'll run it automatically whenever it's needed — and only charge you for those milliseconds."

📦 Example Use Cases

Scenario	How Lambda Helps
🧠 Auto-summarize a document uploaded to S3	Lambda gets triggered on upload, calls Bedrock to summarize
🖼️ Generate image using Bedrock	Lambda listens for user input, calls image model
🔄 Preprocess incoming data	Clean or filter text before storing in DB or S3
🔌 Expose ML models as API	With Lambda + API Gateway
📊 Trigger ETL or pipeline tasks	When a file is uploaded, or daily cron job runs

🔑 Key Features

Feature	Why It Matters for Gen AI
No servers	Zero infrastructure management (AWS handles scaling).
Pay-per-use	$0.20 per 1M requests (cheap for low traffic).
Fast cold starts	~100ms latency (good for text/small-image AI).

🔁 What Triggers a Lambda?

You can configure it to run when:

Trigger Source	Example
S3	File upload (e.g., PDF to summarize)
API Gateway	User hits an endpoint (e.g., `/summarize`)
EventBridge	Run daily at 7PM
DynamoDB	A new record is added
SNS/SQS	New message arrives
Bedrock (via API call)	Trigger LLM model
Manual	You test it in AWS Console or SDK

💻 How Does Lambda Work?

> You upload your code → AWS stores it → You define a trigger → AWS runs the function when triggered

🚀 Gen AI Use Cases

1. **Text Processing**
    - Summarize articles with Hugging Face models (e.g., `distilbart`).

```
import transformers
def lambda_handler(event, context):
    summarizer = transformers.pipeline("summarization")
    return summarizer(event["text"])
```

Cost: ~₹16/month for 10K summaries.

2. **Image Metadata Tagging**
    - Use Lambda + Rekognition to auto-tag uploaded images.
3. **Chatbots**
    - Connect Lambda to API Gateway for a ChatGPT-like API.

⚠️ Limitations

- **15-minute max runtime** (not for heavy training).
- **10GB RAM limit** (use SageMaker/EC2 for big models).
- **Cold starts** (first request may be slow).

🔗 Lambda + Bedrock Example (LLM Call)

```
import boto3, json

client = boto3.client("bedrock-runtime", region_name="us-east-1")

def lambda_handler(event, context):
    user_input = event['prompt']

    response = client.invoke_model(
        modelId="anthropic.claude-3-sonnet-20240229-v1:0",
        body=json.dumps({
            "prompt": f"Human: {user_input}\nAssistant:",
            "max_tokens_to_sample": 100
        }),
        contentType="application/json",
        accept="application/json"
    )

    result = json.loads(response['body'].read())
    return result
```

📦 What This Code Does (High Level):

This code:

- Takes some text from a user (like a question or instruction)
- Sends it to an LLM (like Claude 3 Sonnet) via Amazon Bedrock
- Receives the model's response

- Returns it to the caller (e.g. a frontend or API

`import boto3, json`

- `boto3` is the **official Python SDK for AWS** — it allows Python to talk to AWS services (like S3, EC2, Bedrock, etc.)
- `json` is a built-in Python module for converting data to/from JSON format (used when sending and receiving from Bedrock)

🔷 `client = boto3.client("bedrock-runtime", region_name="us-east-1")`

This creates a **connection to Amazon Bedrock** service.

- `"bedrock-runtime"`: This means we want to **use the model (inference)**, not manage or list it
- `region_name="us-east-1"`: Bedrock only works in certain regions. `us-east-1` is currently one of the main ones.

Think of `client` as your remote control to talk to Bedrock from Python.

🔷 `def lambda_handler(event, context):`

This is the **entry point** of the Lambda function — AWS looks for this function by name.

- `event`: The input data passed into the Lambda (e.g., a dictionary like `{ "prompt": "Tell me a joke" }`)
- `context`: AWS metadata (not needed for now)

🧠 For now: just know AWS calls this function when something triggers Lambda.

🔷 `user_input = event['prompt']`

- This takes the `"prompt"` value from the input data
- If user sent `{ "prompt": "Who is the Prime Minister of India?" }`, then:

```
user_input = "Who is the Prime Minister of India?"
```

⚠️ If this key doesn't exist, it will crash. (We'll fix that later.)

Calling the Model via Bedrock

```
response = client.invoke_model(
    modelId="anthropic.claude-3-sonnet-20240229-v1:0",
    body=json.dumps({
        "prompt": f"Human: {user_input}\nAssistant:",
        "max_tokens_to_sample": 100
    }),
    contentType="application/json",
    accept="application/json"
)
```

✅ `invoke_model()` : This method **sends your input to the chosen model** and gets the output.

`json.dumps()` → 🟢 Converts **Python dictionary → JSON string**

✅ Parameters Explained:

Parameter	Meaning
modelId	Which model to use. This is Claude 3 Sonnet's full ID
body	The actual prompt and settings, converted into JSON
prompt	What you want the model to answer
max_tokens_to_sample	Max words in the output (100 tokens ≈ 75 words)
contentType	You're sending JSON
accept	You want JSON response back

🧠 **The model expects prompts like this:**

```
Human: Tell me a joke.
Assistant:
```

🔷 `result = json.loads(response['body'].read())`

This line:

1. Reads the model's reply (which is like a file-like object)
2. Converts it from JSON format to Python dictionary

Example output:

```
{
  "completion": "Sure! Here's a joke: Why don't scientists trust atoms? Becaus
e they make up everything!"
}
```

Now
`result` is a Python dictionary you can use.

🔷 `return result`

The final line **returns the model's answer** back to whoever triggered this Lambda (like a web app, S3 trigger, or API Gateway).

❌ Beginner Mistakes to Avoid

Mistake	Fix
❌ Writing long-running code (e.g. training a model)	❌ Lambda is NOT for training — use SageMaker or EC2
❌ Using too much memory/time	Max 15 minutes, 10 GB RAM
❌ Forgetting to handle `event` input correctly	✅ Always check keys exist (`event.get()`)
❌ No logs	✅ Use `print()` or `CloudWatch` to debug
❌ Not setting permissions	✅ Add Bedrock, S3, or other required IAM roles